Ders 20 Genişletilmiş Notu: Graflara Giriş, Kapsayan Ağaç ve Kruskal Algoritması

Ders 20 Genişletilmiş Notu: Graflara Giriş, Kapsayan Ağaç ve Kruskal Algoritması

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Graflara Giriş: Düğüm ve Kenar
- 2. Graf Türleri
- 3. Grafların Saklanması: Komşuluk Matrisi ve Komşuluk Listesi
- 4. Yoğun ve Seyrek Graf: Hangi Gösterim Ne Zaman?
- 5. Grafların Kullanım Alanları ve Temel İşlemler
- 6. Matrisin ve Listenin Avantajları
- 7. Kapsayan Ağaç (Spanning Tree)
- 8. En Küçük Kapsayan Ağaç ve Açgözlü Yaklaşım
- 9. Kruskal Algoritması: Birinci Örnek
- 10. Döngü Kontrolü: Ayrık Alt Kümeler ve `belongs` Dizisi
- 11. Kruskal Algoritmasının C Kodu
- 12. Kruskal Algoritması: İkinci Örnek
- Örnek Sorular
- Dersin Toparlaması

1. Graflara Giriş: Düğüm ve Kenar

Bu hafta graflar konusuna geçildi. Hoca dersin planını şöyle verdi: önce graflarla ilgili temel bilgiler, sonra kapsayan ağaç (spanning tree) ve en küçük kapsayan ağaç (minimum spanning tree) kavramları, en sonda da bu problemin nasıl çözüldüğü.

Graf denince aklımıza doğrusal olmayan bir veri yapısı (non-linear data structure) gelmelidir. Dizi ya da bağlı listedeki gibi düz bir sıra yoktur; elemanlar arasında bir ilişki ağı, bir hiyerarşi vardır. Bu hiyerarşi, grafların özel bir türü olan ağaçlarda daha da net görülür.

Graflarda iki temel kavram öne çıkar:

- **Düğüm (vertex, node):** Grafın elemanları.
- **Kenar (edge):** İki düğüm arasındaki bağlantı.

Bir graf G ile gösterilir ve düğüm kümesi V ile kenar kümesi E üzerinden tanımlanır: $G = (V, E)$. Bir grafta birden fazla düğüm ve kenar bulunabilir. Düğüm ve kenarların nasıl yerleştiğine bağlı olarak birçok farklı graf türüyle karşılaşırız.

2. Graf Türleri

Hoca türleri tahtaya numaralandırarak tek tek çizdi.

2.1 Null Graf ve Trivial Graf

Null graf (null graph), hiç kenarı olmayan graftır. Hocanın örneğinde 1, 2, 3, 4 diye dört düğüm vardı ama aralarında hiçbir kenar yoktu. Böyle bir grafa null graf denir.

Trivial graf (trivial graph) ise tek bir düğümden (single vertex) oluşan graftır.

2.2 Yönsüz ve Yönlü Graf

Yönsüz graf (undirected graph) kenarlarında yön bilgisi yoktur. Bir kenar iki düğümü karşılıklı olarak bağlar.

Yönlü graf (directed graph) kenarları oklarla çizilir. Her kenar bir yön bilgisi taşır; bir düğümden diğerine gidilebilir, ama tersi her zaman mümkün değildir.

> **Hocanın vurgusu:** Grafın yönlü ya da yönsüz olması çözümü doğrudan etkiler. Bir graf sorusuyla karşılaştığınızda önce grafın yönlü mü yönsüz mü olduğuna mutlaka bakın.

2.3 Bağlı Graf

Bağlı graf (connected graph) içinde bütün düğümler birbirine, doğrudan ya da başka düğümler üzerinden erişilebilir. Hoca üç graf çizdi: A bağlı bir graftı. B ve C'de ise bazı düğümlere diğerlerinden ulaşmak mümkün değildi. Bu yüzden A'ya bağlı graf, B ve C'ye bağlı olmayan graf (disconnected graph) adı verilir.

2.4 Tam Graf

Tam graf (complete graph), bütün düğümlerin birbirine doğrudan bağlı olduğu graftır. Her düğümden diğer her düğüme doğrudan bir kenar vardır.

Hoca bu yapının bilgisayar ağlarında da bir adı olduğunu söyledi ve sınıfa sordu. Cevap gelmeyince kendisi verdi: **mesh ağ (mesh network)**. Bütün bilgisayarların birbirine doğrudan bağlı olduğu ağ yapısıdır.

2.5 Cycle Graf, Cyclic Graf ve DAG

Cycle graf (cycle graph), kendisi bir döngü (cycle) oluşturan graftır. **Cyclic graf (cyclic graph)** ise kendisi tümüyle bir döngü olmayan ama içinde en az bir döngü bulunan graftır. Aralarındaki fark budur, ama sonuçta ikisinde de döngü vardır.

Çözdüğümüz probleme göre grafın döngülü (cyclic) mi döngüsüz (acyclic) mi olduğunu bilmemiz ve anlayabilmemiz gerekir. Bu, graflarda en önemli özelliklerden biridir.

Bu özellikle ilgili önemli bir tür **yönlü döngüsüz graftır (Directed Acyclic Graph, DAG)**. Hocanın örneğinde yönlü bir graf vardı ve kenarların yönleri izlendiğinde hiçbir döngü oluşmuyordu.

2.6 İki Parçalı Graf

İki parçalı graf (bipartite graph) düğümleri iki kümeye ayrılmış bir graftır. Kenarlar yalnız iki küme arasında, karşılıklı eşleşme biçimindedir. Bir kümenin düğümleri kendi aralarında bağlı değildir.

Hocanın örneği yapay sinir ağlarıydı (neural network). Ardışık iki katman, soldaki A katmanı ve sağdaki B katmanı olsun. A'daki nöronlar B'deki nöronlara bağlıdır, ama aynı katmandaki nöronlar birbirine bağlı değildir. İki katman arasındaki bu ilişki iki parçalı graf olarak tanımlanabilir.

2.7 Ağırlıklı Graf

Hocanın "çok sık kullanacağız" dediği tür **ağırlıklı graftır (weighted graph)**. İki çeşidi vardır: ağırlıklı yönsüz graf ve ağırlıklı yönlü graf.

Önceki örneklerde kenarların ağırlığı yoktu; ilişkiler ağırlıklandırılmamıştı (unweighted graph). Birçok problemde ilişkiler böyle tanımlanır. Ama bazı problemlerde her ilişkinin bir ağırlık derecesi vardır. Hocanın örneği haritalardı: A şehrinden B şehrine olan uzaklığı, yolların mesafelerini kenar ağırlığı yaparak ifade ederiz.

En kısa yol probleminde ve kapsayan ağaçlarda, özellikle en küçük kapsayan ağacı bulurken, hesaplar bu ağırlıklar üzerinden yapılır.

Ağırlıklar genellikle pozitiftir. Ama bazı problemlerde iki düğüm arasındaki ilişki negatif bir değerle de saklanabilir. Bu yüzden ağırlıklı graf denince yalnız pozitif sayılar düşünülmemelidir, ama çoğu problemde ilişkiler pozitif sayılarla tanımlanır.

2.8 Ağaçlar: Grafın Özel Bir Türü

Daha önce görülen ağaçlar, grafların özel bir türüdür. Hoca bir ikili ağaç (binary tree) örneğini graf olarak çizdi ve sordu: Bir grafın ağaç olup olamayacağını söylemek için hangi özelliğe bakmalıyız?

Cevap, biraz önce konuşulan özelliktir: grafın döngü oluşturmaması, yani döngüsüz (acyclic) olması gerekir. Hocanın çizdiği grafta bir döngü vardı, bu yüzden ağaç olma olasılığı yoktu. Döngüdeki bağlardan biri koparılsaydı, teknik olarak bir ağaç elde edilebilirdi.

Bu durumda kökün (root) ne olacağına sizin karar vermeniz gerekir. Herhangi bir düğüm kök olabilir. Bu yüzden bir graftan birden fazla ağaç çıkarma imkânı vardır. Ama döngü varsa ağaçtan söz etmek mümkün değildir.

> **Ders sonrası not:** Hoca burada yalnız döngüsüzlük şartını söyledi. Bir graf ağaç olabilmesi için hem döngüsüz hem de bağlı olmalıdır. Hoca bu iki şartı dersin ilerisinde, kapsayan ağaç tanımında (Bölüm 7) birlikte verdi.

3. Grafların Saklanması: Komşuluk Matrisi ve Komşuluk Listesi

Graflar bilgisayarda iki şekilde gösterilir, yani saklanır:

- **Komşuluk matrisi (adjacency matrix)**
- **Komşuluk listesi (adjacency list)**

Hoca sordu: Hangisi daha iyidir, hangisini kullanmalıyız? Birkaç öğrenci cevap yazdı. Hoca şunu hatırlattı: Bilgisayar biliminde, veri yapıları söz konusu olduğunda, her konuda başarılı mutlak doğru bir veri yapısı yoktur. Her veri yapısının bir kullanım amacı vardır ve duruma (case) göre biri diğerine üstün gelir. Burada da doğrudan “matris daha iyi” ya da “liste daha iyi” diyemeyiz. Hangisinin hangi durumda avantajlı olduğu aşağıda konuşuldu.

3.1 Komşuluk Matrisi

Hocanın örnek grafı yönsüzdür ve 0, 1, 2, 3 diye dört düğümü vardır. Kenarlar: 0-1, 0-2, 1-2, 2-3.

Matriste i. satır ile j. sütunun kesiştiği göz, i ile j arasında kenar varsa 1, yoksa 0 olur. 0’dan 1’e kenar olduğu için o göze 1 yazılır. Graf yönsüz olduğu için 1’den 0’a da 1 yazılmalıdır. Aynı şekilde 0-2, 1-2 ve 2-3 kenarları için iki göz de 1 yapılır.

	0	1	2	3
0	0	1	1	0
1	1	0	1	0
2	1	1	0	1
3	0	0	1	0

Bu matrise bakınca, hangi düğümler arasında bağlantı varsa o gözlerin dolu (1), diğerlerinin 0 olduğu görülür.

> **Hocanın vurgusu:** Yönsüz bir grafın komşuluk matrisi simetrik olmak zorundadır. Simetrik değilse bir bilgi hatası var demektir.

Bu gösterim ağırlıksız graf içindir. Kenarlara ağırlık verildiğinde gözlerde 1 yerine ilgili kenarın ağırlığı yazılır. Bağlantı olmayan gözler ise sonsuz (∞) değeriyle gösterilir.

3.2 Komşuluk Listesi

Aynı graf komşuluk listesiyle şöyle gösterilir: 0, 1'e ve 2'ye komşudur. 1, 0'a ve 2'ye komşudur. 2, 0'a, 1'e ve 3'e komşudur. 3 ise yalnız 2'ye komşudur.

```
0 -> 1 -> 2
1 -> 0 -> 2
2 -> 0 -> 1 -> 3
3 -> 2
```

Her düğüm için bir bağlı liste (linked list) oluşturulur. Düğümün bağlı olduğu düğümlerin bilgisi bu listede saklanır.

3.3 İşlemlerin Karmaşıklığı

Hoca iki gösterimi üç işlem üzerinden karşılaştırdı: kenar ekleme, kenar çıkarma ve ilklendirme (initialization). n düğüm sayısıdır.

İşlem	Komşuluk matrisi	Komşuluk listesi
Kenar ekleme	$O(1)$	$O(1)$
Kenar çıkarma	$O(1)$	Listenin gezilmesi gerekir
İlklendirme	$O(n^2)$	$O(n)$

- **Kenar çıkarma:** Matriste ilgili göz doğrudan sıfırlanır. Listede ise kenarı çıkarmak için listenin gezilmesi gerekir.
- **İlklendirme:** Matris $n \times n$ boyutundadır. Bütün gözlere tek tek erişip ilk değerlerini vermek, yani matrisin üzerinde bir tur atmak gerekir. Bu yüzden $O(n^2)$ olur. Listede her düğüm için bir liste başı ilklendirilir; bu da $O(n)$ olur.

  Ders sonrası not: Altyazıda kenar çıkarma için liste tarafında da "O(1)" geçiyor. Hocanın gerekçesi (tüm listenin gezilmesi) bununla çelişir. Doğrusu: listede kenar silme $O(\deg(v))$, yani en kötü durumda $O(n)$ zaman alır. Altyazıdaki ifade büyük olasılıkla konuşma ya da çözümleme hatasıdır.

4. Yoğun ve Seyrek Graf: Hangi Gösterim Ne Zaman?

Hoca "matris mi liste mi" sorusuna iki anahtar kelimeyle cevap verilebileceğini söyledi: **yoğun graf (dense graph)** ve **seyrek graf (sparse graph)**. Sonra sordu: Bir grafa hangi özelliğine bakarak yoğun ya da seyrek deriz?

Anahtar kavram **kenar sayısıdır**. n düğümümüz olsun. Düğümler arasındaki kenar ilişkileri ne kadar fazla ve yoğunsa graf o kadar yoğun graftır. Hoca bir graf çizdi ve üzerinde yalnız birkaç ilişki gösterdi. Düğümler arasında yoğun bir ikili ilişki olmadığı için bu graf seyrektir. Aynı grafta ilişki sayısını artırırsak yoğun graftan söz etmeye başlarız. Kenar sayısı arttıkça iki gösterimden birini tercih etmemiz gerekir.

Hangisi hangisine uygun? Seyrek graf, matriste sıfırları bol olan bir matris demektir. Çok az bilgi sakladığınız hâlde $n \times n$ alanı boşuna işgal edersiniz. Bu yüzden:

- **Seyrek graf için komşuluk listesi,**
- **Yoğun graf için komşuluk matrisi**

doğru tercihtir.

4.1 Seyrek Graf Örneği ve Yönlülük Sorusu

Hoca bunu A, B, C, D, E, F diye altı düğümlü bir grafla gösterdi. Matrisin 36 gözünden yalnız birkaçı doluydu, geri kalanı 0'dı. Bu bilgiyi 6×6 bir matriste tutmak, boşuna yer kaplayan

bir yapı kullanmak demektir. Liste gösteriminde ise yalnız bağlantısı olan düğümler için bilgi tutulur: A'nın ve B'nin bağlantısı yoktur, bilgi tutulmaz. C'den D'ye ve E'ye bağlantı vardır. Hoca bir yerde bir düğümü kendine bağlı yazdığını fark edip düzeltti.

Sonra sordu: Matrise ve listeye bakınca, seyrek olmasının ötesinde bu graf hakkında ne söyleyebilirsiniz? Bir öğrencinin cevabıyla: **Bu graf yönlüdür.** Bağlantılar incelendiğinde aynı bağlantının karşılıklı bulunmadığı, yalnız tek yönde yer aldığı görülür. Bilgide hata yapılmadığını biliyorsak bu, grafın yönlü olduğunu gösterir.

5. Grafların Kullanım Alanları ve Temel İşlemler

Graflar bugün çok farklı problemlerde kullanılır:

- **Haritalarda rota bulma (routing):** Gerçek hayatta yol bilgisi üzerinden.
- **Sosyal ağlar:** İnsanların birbiriyle bağlantısı, "arkadaşın arkadaşının arkadaşı" gibi ilişkiler graf üzerinde saklanır; kimin kiminle ne tür bağlantısı olduğu bulunur.
- **Bilgisayar ağları:** Bilgisayarların birbiriyle ilişkisi ve ağda yönlendirme.

Hoca günlük hayatta grafları kullandığımız onlarca örnek olduğunu, birçok problemi graf veri yapısıyla çözdüğümüzü ve ilerideki derslerde bunlarla daha sık karşılaşılacağını söyledi.

Graf üzerinde yapılabilecek işlemler:

- Yeni bir düğüm ekleme.
- Bir düğümü ya da kenarı silme.
- Graf üzerinde arama yapma.
- Graf üzerinde gezinme (traversal).

Ağaçlarda gezinirken inorder, preorder, postorder gibi düğüm öncelikleri vardı ve bu önceliklere göre geziyorduk. Graflarda ise iki ana yaklaşımla gezinilir ve arama yapılır:

- **Derinlik öncelikli arama (Depth First Search, DFS)**
- **Genişlik öncelikli arama (Breadth First Search, BFS)**

Bu iki algoritmanın farkı ilerideki derslerde konuşulacak.

6. Matrisin ve Listenin Avantajları

Aradan sonra hoca iki gösterimin avantaj ve dezavantajlarını yazarak toparladı.

Komşuluk matrisi: Genellikle matris üzerinde saklamayı tercih ederiz; birçok şeyi kolaylaştırır, erişim kolaydır. Ama bir yer maliyeti vardır: daha fazla alana ihtiyaç duyar. Ayrıca yeni bir düğüm eklemek görece daha karmaşıktır.

Komşuluk listesi:

- Daha az yer kaybı demektir.
- Sisteme yeni bir düğüm (vertex) eklendiğinde daha rahat eklenir.
- Bir düğümün komşularıyla ilgili işlemler daha verimli bir sürede tamamlanır.

Hoca, bağlı liste kullanıldığı için komşuluk listesinin görece daha az tercih edildiğini, matrisin kolay erişilebilir olduğunu söyledi. Ama bağlı liste yapısının birçok avantajı vardır; özellikle alan kullanımında, hele seyrek graf söz konusuysa, ciddi avantaj sağlar.

> **Ek bilgi (derste anlatılmadı):** Matrisin hızlı olduğu tipik işlem "u ile v arasında kenar var mı?" sorusudur; tek göze bakılarak $O(1)$ zamanda cevaplanır. Listede bunun için u'nun listesi gezilir. Buna karşılık bir düğümün bütün komşularını gezmek matriste her zaman n göze bakmayı gerektirir, listede ise yalnız komşu sayısı kadar adım sürer. Toplam bellek matriste $O(n^2)$, listede $O(n+m)$'dir (m kenar sayısı).

7. Kapsayan Ağaç (Spanning Tree)

Bugünün ilk temel graf problemi kapsayan ağaç problemidir ve bu problemin içinde en küçük kapsayan ağaç aranacaktır.

Hoca slayttaki graflar üzerinden iki temel özelliğe dikkat çekti: **bağlılık (connectivity)** ve **döngüsüzlük (acyclic yapı)**. Kapsayan ağaçtan söz ediyorsak bu iki özelliğin ikisi de sağlanmalıdır.

7.1 Döngülü Graf

Slayttaki ilk graf dört düğümlüdür. Bu grafta döngüsüz bir yapıdan söz edilemez. Hoca grafta iki küçük döngü olduğunu, dıştaki ana döngüyü de sayarsak toplam üç döngü bulunduğunu söyledi. Bu gösterimle kapsayan ağaçtan söz etmek mümkün değildir. Ama grafın içinden, onun altında oluşabilen kapsayan ağaçlar çıkarılabilir.

Bağlılık açısından ise bu graf bağlıdır: bir düğümden başladığınızda istediğiniz düğüme erişebilirsiniz.

7.2 Bağlı Olmayan Yapı ve Orman

Yandaki grafta A, B, C, D düğümlerine ek olarak bir de E düğümü vardır. E'den diğer düğümlere ya da A, B, C'den E'ye erişmek mümkün değildir. Bu yüzden bu yapı bağlı değildir.

Bu yapıyı ağaç olarak ele alırsak iki ayrı ağaç vardır. Birden fazla ağaçtan oluşan bu yapıya **orman (forest)** denir. E olmasaydı yapı bir ağaç olacaktı. E'ye ek olarak bir F düğümü de olsaydı, yine F'den A, B, C, D'ye erişilemeyeceği için bağlılıktan söz edemezdik. Bağlılık kolayca tespit edilir; ama döngüsüzlüğün tespiti de en az onun kadar önemlidir.

7.3 Bir Graftan Birden Fazla Kapsayan Ağaç

Hoca slaytta döngülü dört düğümlü graftan çıkarılan alt grafları gösterdi. Her biri hem bağlıdır (tüm düğümlere erişilir) hem de döngü içermez. Bu yüzden her biri bir kapsayan ağaç adayıdır.

- Bir ağaçta B-C ve C-D bağlantıları çıkarılmıştı.
- Bir başka ağaçta A-B ve B-C bağlantıları çıkarılmıştı.

Hoca bu ağaçların gezilme sıralarını da söyledi: ABCD, CABD, ACDB ve ABDC. Bunların hepsi bir kapsayan ağaç oluşturur. Yani bir graftan birden fazla kapsayan ağaç elde edilebilir.

Hocanın söylediği çıkarmalar ve gezilme sıraları birlikte düşünüldüğünde grafın kenarları A-B, A-C, B-C, B-D, C-D olarak çıkar (ara adım eklendi). Bu kenar kümesi hocanın saydığı üç döngüyle de uyumludur: A-B-C, B-C-D ve dıştaki A-B-D-C.

> **Ders sonrası not:** Hoca "grafların altında n tane kapsayan ağaç oluşturma şansınız var" dedi; bu kesin bir sayı değil, "birden fazla" anlamında söylenmiştir. Yukarıdaki kenar kümesiyle bu grafın bütün 3 kenarlı alt kümeleri python ile tek tek denendiğinde tam 8 kapsayan ağacı olduğu bulundu; slaytta bunlardan dördü gösterilmişti.

8. En Küçük Kapsayan Ağaç ve Açgözlü Yaklaşım

8.1 Tanım

Biz bu aday kapsayan ağaçlar içinden **en küçüğünü** arıyoruz. En küçükten söz ediyorsak düğümler arasındaki kenarların bir ağırlığı olmalıdır. Bu ağırlıklar üzerinden tüm düğümlere en az eforla ulaşılmalıdır. Kapsayan ağaçlar arasından toplam ağırlığı en küçük olana **en küçük kapsayan ağaç (minimum spanning tree, MST)** denir. Bazen birden fazla kapsayan ağaç aynı en küçük değere sahip olabilir; o zaman birden fazla MST vardır.

En küçük kapsayan ağacı bulmak için iki temel algoritma vardır: **Prim algoritması** ve **Kruskal algoritması**.

8.2 MST Nerede İşe Yarar?

Hoca sınıfa MST'nin nerede işe yarayabileceğini sordu ve kendisi gezgin satıcı problemi (Traveling Salesman Problem) örneğini verdi. Bir satıcının belirli şehirlere uğraması gerekir ve ona bir liste verilmiştir. Bu şehirleri en az eforla nasıl ziyaret edeceğine dair bir plan yaparken en küçük kapsayan ağaç algoritmalarından faydalanılır.

Ders sonrası not: MST gezgin satıcı problemini doğrudan çözmez ve bir gezi rotası (tur) da değildir; yalnız tüm düğümleri en az toplam ağırlıkla birbirine bağlayan kenar kümesidir. Gezgin satıcı problemi NP-zor bir problemdir. MST, kenar ağırlıkları üçgen eşitsizliğini sağlayan (metrik) gezgin satıcı probleminde en fazla 2 kat kötü bir yaklaşık tur kurmak için kullanılır. Hocanın "planlama yaparken faydalanırız" ifadesi bu anlamda doğrudur.

8.3 Dört Düğümlü Ağırlıklı Örnek

Hoca bu kez kenarları ağırlıklı dört düğümlü (A, B, C, D) bir graf gösterdi. Kenarlar A-B, A-C, A-D ve C-D'dir. Bu grafın üç kapsayan ağacı vardır:

Ağaç	Kenarlar	Toplam ağırlık
T_1	A-B, A-D, C-D	6
T_2	B-A, A-C, C-D	9
T_3	B-A, A-C, A-D	8

Kenar ağırlıkları kısa nottaki slayt bilgisine göre: A-B = 1, A-C = 5, A-D = 2, C-D = 3. Buna göre:

$$w(T_1) = 1 + 2 + 3 = 6, \quad w(T_2) = 1 + 5 + 3 = 9, \quad w(T_3) = 1 + 5 + 2 = 8$$

En küçük değer T_1'dedir; bu grafın MST'si T_1'dir ve ağırlığı 6'dır. Hesap python ile doğrulandı.

Hoca burada önemli bir noktaya dikkat çekti: Biz bunları çizerek, görerek hesaplıyoruz; analitik olarak hesaplamıyoruz. Bilgisayarın ise daha karmaşık graflarda bunu analitik olarak hesaplaması gerekir. Bunun yolu algoritmalardır.

8.4 Nasıl Bir Yaklaşım? Açgözlü Teknik

Hoca bir graf daha çizdi ve sordu: Prim ya da Kruskal'a takılmadan, bütün kapsayan ağaçlar içinden en düşük ağırlıklı olanı bulmak için nasıl bir yaklaşım benimsersiniz?

Burada **açgözlü teknik (greedy technique)** kullanılır. Açgözlü algoritmalar listelendiğinde Prim ve Kruskal bu teknik altında yer alır.

Öğrencilerden iki cevap geldi:

- Bir başlangıç düğümü seçip küçük ağırlıklı kenarları seçe seçe gitmek.
- "Hep en kısa yoldan gitmek": Bir öğrenci bunu örnekle açıkladı: C'den D'ye 3, C'den B'ye 2 ise daha az olduğu için B'ye gideriz. Yani en kısa kenar tercih edilir.

Hoca bu cevaplardan yola çıkarak iki temel yaklaşımı tanımladı:

- **Kruskal yaklaşımı:** Kenarları küçükten büyüğe sıralarım. Her seferinde listedeki en küçük kenarı alır, kapsayan ağaca eklerim. Bunları ekleye ekleye MST'yi oluştururum.
- **Prim yaklaşımı:** Kendime bir başlangıç düğümü seçerim. Bu düğümden başka düğümlere giden olası yollar içinden en kısa olanı tercih ederim.

Bu derste Kruskal işlendi; Prim bir sonraki derse kaldı.

8.5 Neden "Açgözlü"?

Bir algoritma her adımda, o andaki seçenekler içinden en iyisini (en optimal olanı) alarak ilerliyor ve bu seçimler sonunda toplamda en iyi sonuca ulaştırıyorsa ona **açgözlü algoritma (greedy algorithm)** denir. Her alt problemde elinizdeki en iyi seçeneği seçip ilerlersiniz ve en iyi sonuca varırsınız. Kruskal'da her adımda en küçük kenarı almak bu tekniğin bir örneğidir.

9. Kruskal Algoritması: Birinci Örnek

Hoca slayttaki bir grafla Kruskal'ı adım adım çalıştırdı. Graf altı düğümlüdür (A-F) ve on kenarı vardır. Hoca kenar sayısının düğüm sayısından fazla olduğunu, bu yüzden bunun teknik olarak yoğun bir graf olduğunu ve matriste saklanabileceğini söyledi.

9.1 Kenarları Listeleme ve Sıralama

İlk iş, bütün kenarları uzunluklarıyla birlikte bir listeye koymak, sonra bu listeyi sıralamaktır. Sıralı listede birinci sırada B-C, ikinci sırada E-F, üçüncü sırada A-B vardır; liste on kenara kadar devam eder. Algoritma bu sıralı listeden çalışır.

9.2 Adımlar

Ana kural şudur: Listeden her seferinde sıradaki en küçük kenar alınır. Ama alınacak kenar, o ana kadar seçilen kenarlarla bir **döngü oluşturuyorsa** o kenar alınmaz. Hoca bunu sınıfa sordu ("Hangi durumda sıradaki kenar en küçük olmasına rağmen onu almamalıyız?") ve bir öğrencinin cevabıyla vurguladı.

> **Hocanın vurgusu:** Aldığımız kenar bir döngü oluşturursa, o kenarı almaktan vazgeçeriz. Bu algoritmanın bütün temel prensibi budur.

Adım	Kenar	Ağırlık	Sonuç	O andaki seçilmiş kenarlar
1	B-C	1	Alındı	B-C
2	E-F	2	Alındı	B-C, E-F
3	A-B	3	Alındı	A-B, B-C, E-F
4	B-F	4	Alındı	A-B, B-C, B-F, E-F
5	C-F	4	Döngü (B-C-F), alınmadı	
6	A-F	5	Döngü (A-B-F), alınmadı	
7	D-F	5	Alındı	A-B, B-C, B-F, D-F, E-F
8-10	A-E, C-D, D-E	6, 6, 8	Bakılmadı	

- B-F ile C-F'nin ağırlığı eşittir. Hoca böyle durumlarda genelde harf önceliği olan kenarın alındığını söyledi; önce B-F alındı.
- C-F eklenseydi B-C-F döngüsü oluşurdu. Bu kenar seçilen kenarlarla birleşince artık kapsayan ağaç oluşturma ihtimali ortadan kalkar; bu kontrol yapılır ve kenar atlanır.
- A-F de aynı sebeple alınamaz: A-B-F döngüsü oluşur.
- D-F eklendiğinde döngü oluşmaz.

9.3 Durma Noktası ve Sonuç

Hoca sordu: Durma noktası neresidir? Cevabı: Aldığımız kenarlarla bütün düğümlere dokunabildiysek geri kalan kenarlara bakmaya gerek kalmaz. D-F'den sonra A, B, C, D, E ve F'nin hepsine dokunulmuştur; kalan kenarlar kontrol edilmez.

Seçilen kenarların ağırlıkları 1, 2, 3, 4 ve 5'tir:

$$w(\text{MST}) = 1 + 2 + 3 + 4 + 5 = 15$$

Bu grafın MST değeri 15'tir. Altı düğümlü grafta MST'de $6 - 1 = 5$ kenar vardır. Sonuç python ile doğrulandı.

> **Ders sonrası not:** "Bütün düğümlere dokunulduysa dur" ölçütü genel olarak doğru değildir. Seçilen kenarlar bütün düğümlere dokunduğu hâlde birden fazla ayrık parça kalabilir. Örneğin dört düğümde önce 0-1, sonra 2-3 seçilirse dört düğüme de dokunulmuştur ama 1-2 gibi bir kenar daha gerekir. Doğru ölçüt, n düğümlü grafta $n - 1$ kenar seçilmiş olması ya da bütün düğümlerin tek bir kümede birleşmesidir. Hoca ikinci örnekte (Bölüm 12) bu ikinci ölçütü de söyledi. Bu derste çözülen iki örnekte iki ölçüt aynı adımda sağlandığı için sonuç değişmez.

Ders sonrası not: Hoca sonucu "bütün düğümleri gezerek bütün grafa gidebilmenin yolu 15 birimlik yol kat etmek" diye yorumladı. MST bir gezi yolu değildir; 15, tüm düğümleri birbirine bağlayan kenarların toplam ağırlığıdır. Bir ağacı uçtan uca dolaşan bir yol, kenarlardan geri dönmek zorunda kalabileceği için genelde daha uzundur.

9.4 Algoritmanın Özeti

Hocanın toparladığı biçimiyle Kruskal:

1. Kenarları bir listeye koy ve küçükten büyüğe sırala.
2. En küçük kenardan başlayarak sıradaki kenarı al.
3. Bu kenar, seçilmiş kenarlarla döngü oluşturuyor mu diye kontrol et.
4. Döngü oluşturmuyorsa kenarı MST listesine ekle; oluşturuyorsa atla. Sonraki en küçük kenara geç.
5. Tüm düğümlere ulaşıldığında dur.

10. Döngü Kontrolü: Ayrık Alt Kümeler ve `belongs` Dizisi

Hoca, Kruskal'ı sözlü anlattıktan sonra döngü kontrolünün bilgisayarda nasıl yapıldığını gösterdi. Döngü olup olmadığı, alt kümelerin (subset) birbiriyle ilişkisi kullanılarak kontrol edilir.

Her düğümün hangi kümeye ait olduğunu tutan bir dizi vardır. Kodda adı `belongs`'tur. Aynı kümedeki düğümler aynı değeri taşır.

10.1 A Örneği: Aynı Kümedeki İki Düğüm

İlk örnekte A, B, C, D, E düğümleri döngü oluşturmadan birbirine bağlanmıştır ve hepsinin değeri 1'dir. Bu sırada bir x kenarı gelir ve U ile V düğümlerini bağlamak ister. İki düğümün değeri aynıdır (ikisi de 1). Bu, iki düğümün zaten aynı ağaçta olduğunu gösterir; aralarına kenar eklenirse döngü oluşur. Bu yüzden x kenarı MST'ye eklenmez.

10.2 B Örneği: İki Ayrık Alt Kümenin Birleşmesi

İkinci örnekte iki ayrık alt küme (disjoint subset) vardır:

- A, B, C, D, E, F, G düğümlerinin değeri 1'dir.
- H, I, J, K düğümlerinin değeri 2'dir.

U ile V'yi bağlayan bir kenar seçildiğinde, U'nun değeri 1, V'nin değeri 2'dir. Değerler eşit olmadığı için kenar alınır ve iki taraf bağlanır. Bağlantı kurulduğu anda iki kümeden biri, değerlerini diğer kümenin değeriyle günceller. Hocanın örneğinde H, I, J, K'nin değerleri 1 yapıldı. Böylece iki ayrık alt kümeden tek bir alt küme oluşur.

Bundan sonra dışarıda, örneğin değeri 3 olan başka bir alt küme varsa ve onunla bağlantı kurulursa, 1 ile 3 farklı olduğu için o küme de bu büyük kümeyle birleştirilir.

Ama E ile G düğümlerini birleştirecek bir kenar gelirse, E'nin değeri de G'nin değeri de 1'dir. Bu kenar döngü oluşturacağı için alt kümeye eklenmez; sistem onu almamayı tercih eder.

Kısacası: Kenarın iki ucunun küme değeri **farklıysa** kenar alınır ve iki küme birleştirilir; **aynıysa** kenar döngü oluşturur ve alınmaz. Bu mantık iki ya da n tane ayrık alt kümenin birleştirilmesinde aynı şekilde işler.

> **Ek bilgi (derste anlatılmadı):** Bu yapının genel adı ayrık küme (disjoint set) ya da union-find veri yapısıdır. Derste gösterilen dizi yönteminde `find` $O(1)$, birleştirme ise bütün diziyi gezdiği için $O(n)$ sürer. Kruskal'ın toplam süresi kenarları sıralamanın maliyeti olan $O(m \log m)$ ile birleştirme maliyetinin toplamıdır. Kümeleri ağaç olarak tutup rank ve yol sıkıştırma (path compression) kullanan sürümlerde birleştirme işlemleri neredeyse sabit zamana iner.

11. Kruskal Algoritmasının C Kodu

Son bölümde hoca ekranda Kruskal'ın yazılmış bir C kodunu paylaştı ve kod üzerinden neye ihtiyaç olduğunu anlattı.

11.1 Veri Yapıları

- **`edge` yapısı:** Bir kenarın iki düğümünü (`u`, `v`) ve ağırlığını (`w`) tutar.
- **`edge_list` yapısı:** Kenarların eklendiği, `edge` tipinde bir dizi (`data`) ve listenin uzunluğunu tutan `n`.
- **`elist`:** Bu tipten yaratılan, graftaki bütün kenarları tutan liste.
- **`Graph`:** Graf komşuluk matrisi olarak saklanır.
- **`spanlist`:** MST oluşturulurken adım adım seçilen kenarların eklendiği liste.

Fonksiyonlar:

- `kruskalAlgo`: Kruskal'ın ana prensiplerini çalıştırır.
- `find`: Eklenecek kenarın düğümlerinin küme değerini bulur.
- `applyUnion`: İki kümeyi birleştirir.
- `sort` ve `print`: Kenarları sıralar ve sonucu ekrana yazdırır.

`main` içinde $n = 6$ verilmişti; düğümlerin değerleri ve kenar uzunlukları komşuluk matrisine elle yazılıyordu. Ardından `kruskalAlgo` çağrılıyor, sonra sonuçlar ekrana yazdırılıyordu.

11.2 `kruskalAlgo` Adım Adım

1. `belongs` dizisi tanımlanır. Hoca önce tüm değerlerin sıfırla başlatıldığını söyledi, sonra "burası önemli" diyerek düzeltti (aşağıda 4. adım).
2. `elist.n = 0` yapılır, çünkü henüz hiç kenar eklenmedi.
3. Komşuluk matrisi gezilir. Bir gözdeki değer sıfırdan farklıysa satır numarası birinci düğüm (`u`), sütun numarası ikinci düğüm (`v`), gözdeki değer kenarın ağırlığı (`w`) olarak `elist`'e eklenir ve `elist.n++` ile kenar sayısı bir artırılır.
4. Liste `sort` ile sıralanır.
5. Her düğüm ayrı bir değer alır: `belongs[i] = i`. Yani her düğüm başlangıçta kendi kümesindedir; henüz birleştirme yapılmadığı için hepsinin değeri farklıdır.
6. `spanlist.n = 0`: MST'de henüz kenar yoktur.
7. Sıralı listenin başından başlanarak her kenar için iki ucun değeri `find` ile `belongs` dizisinden alınır. Örneğin 1. ve 3. düğüm geldiyse değerleri 1 ve 3'tür. Değerler farklıysa kenar `spanlist`'e eklenir, `spanlist.n` bir artırılır ve iki küme `applyUnion` ile birleştirilir.
8. `applyUnion`, `belongs` dizisinde `c2` değerine sahip bütün elemanları `c1` yapar. 1 ile 3 birleşince 3'ün değeri 1 olur.
9. Sonraki turda, örneğin 3-5 kenarı gelirse: 3'ün değeri artık 1'dir, 5'in değeri 5'tir. Farklı oldukları için birleştirilirler. Her yeni kenarda iki ucun değeri farklıysa kümeler birleştirilerek birleşimler (union) oluşturulur.

11.3 Kod

```c
#include <stdio.h>

#define MAX 30

typedef struct edge {
  int u, v, w;
} edge;

typedef struct edge_list {
  edge data[MAX];
  int n;
} edge_list;

edge_list elist;
```

```c
int Graph[MAX][MAX], n;
edge_list spanlist;

void kruskalAlgo(void);
int find(int belongs[], int vertexno);
void applyUnion(int belongs[], int c1, int c2);
void sort(void);
void print(void);

void kruskalAlgo(void) {
  int belongs[MAX], i, j, cno1, cno2;
  elist.n = 0;

  /* Matrisin alt üçgeninden kenar listesini çıkar */
  for (i = 1; i < n; i++)
    for (j = 0; j < i; j++) {
      if (Graph[i][j] != 0) {
        elist.data[elist.n].u = i;
        elist.data[elist.n].v = j;
        elist.data[elist.n].w = Graph[i][j];
        elist.n++;
      }
    }

  sort(); /* kenarları ağırlığa göre sırala */

  for (i = 0; i < n; i++)
    belongs[i] = i; /* her düğüm ayrı kümede */

  spanlist.n = 0;

  for (i = 0; i < elist.n; i++) {
    cno1 = find(belongs, elist.data[i].u);
    cno2 = find(belongs, elist.data[i].v);

    if (cno1 != cno2) { /* farklı kümeler: döngü yok */
      spanlist.data[spanlist.n] = elist.data[i];
      spanlist.n = spanlist.n + 1;
      applyUnion(belongs, cno1, cno2);
    }
  }
}

int find(int belongs[], int vertexno) {
  return (belongs[vertexno]);
}

void applyUnion(int belongs[], int c1, int c2) {
  int i;
  for (i = 0; i < n; i++)
    if (belongs[i] == c2)
      belongs[i] = c1;
}

/* Derlenebilir olsun diye eklendi: kabarcık sıralaması (ağırlığa göre) */
void sort(void) {
  int i, j;
  edge temp;
  for (i = 1; i < elist.n; i++)
```

```
    for (j = 0; j < elist.n - 1; j++)
      if (elist.data[j].w > elist.data[j + 1].w) {
        temp = elist.data[j];
        elist.data[j] = elist.data[j + 1];
        elist.data[j + 1] = temp;
      }
}

/* Derlenebilir olsun diye eklendi: MST kenarlarını ve toplamı yazdır */
void print(void) {
  int i, cost = 0;
  for (i = 0; i < spanlist.n; i++) {
    printf("%d - %d : %d\n", spanlist.data[i].v, spanlist.data[i].u,
           spanlist.data[i].w);
    cost = cost + spanlist.data[i].w;
  }
  printf("Spanning tree cost: %d\n", cost);
}

/* Test verisi: dersteki ikinci örnek (9 düğüm, 14 kenar) */
int main(void) {
  int k;
  int e[14][3] = {{6, 7, 1}, {2, 8, 2}, {5, 6, 2}, {0, 1, 4}, {2, 5, 4},
                  {6, 8, 6}, {2, 3, 7}, {7, 8, 7}, {0, 7, 8}, {1, 2, 8},
                  {3, 4, 9}, {4, 5, 10}, {1, 7, 11}, {3, 5, 14}};
  n = 9;
  for (k = 0; k < 14; k++) {
    Graph[e[k][0]][e[k][1]] = e[k][2];
    Graph[e[k][1]][e[k][0]] = e[k][2];
  }
  kruskalAlgo();
  print();
  return 0;
}
```

Kod `gcc -Wall` ile uyarısız derlendi ve çalıştırıldığında şu çıktıyı verdi:

```
6 - 7 : 1
5 - 6 : 2
2 - 8 : 2
0 - 1 : 4
2 - 5 : 4
2 - 3 : 7
1 - 2 : 8
3 - 4 : 9
Spanning tree cost: 37
```

Toplam 37, Bölüm 12'deki el çözümüyle aynıdır. Tek fark, ağırlığı 8 olan iki kenardan (0-7 ve 1-2) kodun 1-2'yi, hocanın ise 0-7'yi seçmesidir. İkisi de aynı adımda iki ayrı kümeyi birleştirir ve aynı toplam ağırlığı verir; bu grafın ağırlığı 37 olan iki farklı MST'si vardır. Hangisinin seçileceği eşit ağırlıklı kenarların sıralamadaki yerine bağlıdır.

Kodda matrisin yalnız alt üçgeni ($j < i$) gezildiği için yönsüz grafın her kenarı listeye bir kez girer.

11.4 Hocanın İyileştirme Önerisi

Hoca bu kodun eksiklerinden, iyileştirilebilir yönlerinden birini söyledi: Kod, MST tamamlansa bile sıralı listedeki bütün kenarların üzerinden geçer. Kenar ekledikçe ulaşılan düğümler ayrı bir dizide işaretlenebilir. Bütün düğümlere ulaşıldığı anda sistem işlemini kendiliğinden sonlandırır; kalan kenarlara bakılmaz.

Ders sonrası not: Erken durma fikri doğrudur, ama “bütün düğümler işaretlendi” ölçütü tek başına yetmez (bkz. Bölüm 9.3’teki not). Güvenli ölçüt `spanlist.n == n - 1` olmasıdır ya da hocanın ikinci örnekte söylediği gibi `belongs` dizisindeki bütün değerlerin aynı olmasıdır.

12. Kruskal Algoritması: İkinci Örnek

Hoca öğrencilere bir örnek daha isteyip istemediklerini sordu; istek üzerine yeni bir örnek çizdi. Graf dokuz düğümlüdür (0-8) ve 14 kenarı vardır.

12.1 Kenarların Sıralanması

Hoca kenarları küçükten büyüğe şöyle sıraladı:

Sıra	Kenar	Ağırlık
1	6-7	1
2	5-6	2
3	2-8	2
4	0-1	4
5	2-5	4
6	6-8	6
7	2-3	7
8	7-8	7
9	0-7	8
10	1-2	8
11	3-4	9
12	4-5	10
13	1-7	11
14	3-5	14

Hoca kenarları bir kez daha sayarak 14 kenarın hepsinin listede olduğunu kontrol etti.

12.2 `belongs` Dizisiyle Adım Adım Çözüm

Hoca bu kez `belongs` dizisini de tahtaya çizdi. Başlangıçta her düğüm kendi değerini alır: 0, 1, 2, …, 8. Birleştirmede hangi kümenin değerinin korunacağını hoca adım adım kendisi seçti; aşağıdaki tablo hocanın seçimlerini izler.

Adım	Kenar	Uçların değeri	Sonuç	`belongs` (0..8) sonrası
0	-	-	Başlangıç	0 1 2 3 4 5 6 7 8
1	6-7 (1)	6, 7	Farklı, alındı	0 1 2 3 4 5 7 7 8
2	5-6 (2)	5, 7	Farklı, alındı	0 1 2 3 4 5 5 5 8
3	2-8 (2)	2, 8	Farklı, alındı	0 1 2 3 4 5 5 5 2
4	0-1 (4)	0, 1	Farklı, alındı	1 1 2 3 4 5 5 5 2
5	2-5 (4)	2, 5	Farklı, alındı	1 1 2 3 4 2 2 2 2
6	6-8 (6)	2, 2	Aynı, döngü, alınmadı	1 1 2 3 4 2 2 2 2
7	2-3 (7)	2, 3	Farklı, alındı	1 1 2 2 4 2 2 2 2
8	7-8 (7)	2, 2	Aynı, döngü, alınmadı	1 1 2 2 4 2 2 2 2
9	0-7 (8)	1, 2	Farklı, alındı	1 1 1 1 4 1 1 1 1
10	1-2 (8)	1, 1	Aynı, döngü, alınmadı	1 1 1 1 4 1 1 1 1
11	3-4 (9)	1, 4	Farklı, alındı	4 4 4 4 4 4 4 4 4
12-14	4-5, 1-7, 3-5	-	Bakılmadı	

Adımlarla ilgili hocanın açıklamaları:

- **6-7:** Değerler 6 ve 7, farklı. Biri diğerine eşitlenir; ikisi de 7 oldu.
- **5-6:** 6’nın karşılığı 7, 5’inki 5; farklı. Yeni gelen değerden dolayı grubun hepsi 5 oldu.

- **2-8:** 8, 2 yapıldı.
- **0-1:** Farklı oldukları için birleştirildi; ikisi de 1 oldu.
- **2-5:** Ağırlığı 4. Görsel olarak da döngü oluşmuyor; 5 değerli düğümler 2 yapıldı.
- **6-8:** Değerleri aynı. Görsel olarak da birleştirilirse döngü oluşur; elendi.
- **2-3:** Değerler farklı. Hoca önce "bütün 2'leri 3 yaparım" dedi, sonra "yeni gelen 3'ü 2 yaparım" diye düzeltti.
- Bu noktada hoca bir ara gözlem yaptı: Alınan düğümler ayrı bir dizide işaretleniyor olsaydı 0, 1, 2, 3, 5, 6, 7, 8 alınmış, yalnız 4 alınmamış olurdu; bu yüzden aramaya devam edilir.
- **7-8:** `belongs` dizisinde değerleri aynı; alınmadı.
- **0-7:** 0'ın değeri 1, 7'nin değeri 2. Hoca sordu: "0 ile 7'yi birleştirmem gerekiyor mu, o yola ihtiyaç duyuyor muyum?" Cevap evet; iki ayrı grubun birleşmesi gerekir. Kalan bütün değerler 1 yapıldı. Artık yalnız 4'e ulaşılmamıştır.
- **1-2:** Değerler 1 ve 1, aynı; alınmadı.
- **3-4:** Değerler farklı; kenar eklendi ve hepsi 4'e dönüştürüldü.

Hoca burada önemli bir gözlem yaptı: Bütün düğümler aynı değeri aldığında aslında hepsine ulaşılmış olur. Ek bir diziye ihtiyaç duymadan da `belongs` dizisini kontrol ederek işin bittiği anlaşılır. Kalan 4-5, 1-7 ve 3-5 kenarlarına bakmaya gerek kalmaz.

Seçilen kenarlar 6-7, 5-6, 2-8, 0-1, 2-5, 2-3, 0-7, 3-4'tür. Sayıları $9 - 1 = 8$'dir. Hoca toplam ağırlığı söylemedi; tablodan hesaplanırsa:

$$w(\text{MST}) = 1 + 2 + 2 + 4 + 4 + 7 + 8 + 9 = 37$$

Bu sonuç python ile Kruskal çalıştırılarak ve 8 kenarlı bütün alt kümeler tek tek denenerek doğrulandı; derlenen C kodu da aynı toplamı verdi.

> **Ek bilgi (derste anlatılmadı):** Birleştirmede hangi kümenin değerinin korunduğu sonucu değiştirmez; önemli olan, birleşen iki kümenin bütün elemanlarının aynı değeri almasıdır. Derste gösterilen C kodu her zaman `c2` değerli elemanları `c1` yapar, hoca ise elle çözümde yönü adım adım kendisi seçti. İkisi de aynı kenarları kabul ve reddeder.

Örnek Sorular

Bu derste ekrandan alınmış hazır kare yoktur; aşağıdaki sorular derste sözlü sorulan ve çözülen sorulardır.

Soru 1 (derste sözlü, 00:15:26): Tüm düğümlerin birbirine doğrudan bağlı olduğu tam grafın bilgisayar ağlarındaki karşılığı olan ağ tipi nedir?

> **Çözüm:** Mesh ağ (mesh network). Bütün bilgisayarlar birbirine doğrudan bağlıdır.

Soru 2 (derste sözlü, 00:21:59): Bir grafın ağaç oluşturup oluşturamayacağını söylemek için hangi özelliğine bakılır?

> **Çözüm:** Grafın döngü içermemesi (acyclic olması) gerekir. Döngü varsa ağaçtan söz edilemez; döngüdeki bir kenar koparılırsa ağaç elde edilebilir. Kapsayan ağaç tanımında hoca ikinci şart olarak bağlılığı da ekledi.

Soru 3 (derste sözlü, 00:23:34): Grafı saklamak için komşuluk matrisi mi, komşuluk listesi mi daha iyidir?

> **Çözüm:** Mutlak bir cevap yoktur; duruma bağlıdır. Seyrek graf için komşuluk listesi, yoğun graf için komşuluk matrisi doğru tercihtir. Seyrek grafı matriste tutmak, çoğu sıfır olan $n \times n$ alanı boşa harcar.

Soru 4 (derste sözlü, 00:30:48): Bir grafa neye bağlı olarak yoğun (dense) ya da seyrek (sparse) deriz?

> **Çözüm:** Kenar sayısına bağlı olarak. Düğümler arasındaki kenar ilişkileri ne kadar fazlaysa graf o kadar yoğundur.

Soru 5 (derste sözlü, 00:35:29): Altı düğümlü (A-F) seyrek grafın matrisine ve listesine bakınca, seyrek olmasının ötesinde graf hakkında ne söylenebilir?

> **Çözüm:** Graf yönlüdür. Aynı bağlantı karşılıklı olarak bulunmuyor, yalnız tek yönde yer alıyor; matris simetrik değildir.

Soru 6 (derste sözlü, 01:04:28): En küçük kapsayan ağacı bulmak nerede işimize yarar?

> **Çözüm:** Hocanın verdiği örnek gezgin satıcı problemidir: Satıcının şehirleri en az eforla ziyaret edeceği bir plan kurulurken MST algoritmalarından faydalanılır. MST'nin bu problemi tek başına çözmediği, yaklaşık çözüm için kullanıldığı Bölüm 8.2'deki notta açıklandı.

Soru 7 (derste sözlü, 01:08:43): Prim ve Kruskal'a takılmadan, bir graftaki kapsayan ağaçlar içinden en düşük ağırlıklı olanı bulmak için nasıl bir yaklaşım benimsersiniz?

> **Çözüm:** Açgözlü (greedy) bir yaklaşım. İki yol vardır: Kenarları küçükten büyüğe sıralayıp her seferinde döngü oluşturmayan en küçük kenarı eklemek (Kruskal) ya da bir başlangıç düğümü seçip ağaçtan dışarı giden en kısa kenarı eklemek (Prim).

Soru 8 (derste sözlü, 01:17:48): Kruskal'da hangi durumda sıradaki kenar en küçük olmasına rağmen alınmaz?

> **Çözüm:** O kenar, seçilmiş kenarlarla bir döngü oluşturuyorsa alınmaz. Bilgisayarda bu, kenarın iki ucunun `belongs` değerinin aynı olmasıyla anlaşılır.

Soru 9 (derste sözlü, 01:20:40): Kruskal algoritmasında durma noktası neresidir?

> **Çözüm:** Hocanın cevabı: Seçilen kenarlarla bütün düğümlere dokunulduysa geri kalan kenarlara bakılmaz. İkinci örnekte hoca, `belongs` dizisindeki bütün değerlerin aynı olmasının da işin bittiğini gösterdiğini söyledi. Güvenli ölçüt $n - 1$ kenar seçilmesi ya da tek bir kümenin kalmasıdır (Bölüm 9.3'teki not).

Soru 10 (derste sözlü, 01:13:03 ve 01:15:37): Altı düğümlü, on kenarlı grafın en küçük kapsayan ağacını Kruskal ile bulun.

> **Çözüm:** Bölüm 9'da adım adım çözüldü. Alınan kenarlar B-C (1), E-F (2), A-B (3), B-F (4), D-F (5); C-F ve A-F döngü oluşturduğu için alınmadı. MST ağırlığı $1 + 2 + 3 + 4 + 5 = 15$.

Soru 11 (derste sözlü, 01:54:14): Dokuz düğümlü (0-8), 14 kenarlı grafın en küçük kapsayan ağacını Kruskal ve `belongs` dizisiyle bulun. Kenarlar: 0-1 (4), 0-7 (8), 1-2 (8), 1-7 (11), 2-3 (7), 2-5 (4), 2-8 (2), 3-4 (9), 3-5 (14), 4-5 (10), 5-6 (2), 6-7 (1), 6-8 (6), 7-8 (7).

> **Çözüm:** Kenarların seçimi derste adım adım yapıldı (Bölüm 12.2). Alınan kenarlar 6-7, 5-6, 2-8, 0-1, 2-5, 2-3, 0-7, 3-4; reddedilenler 6-8, 7-8, 1-2.
>
> **Çözüm (derste verilmedi):** MST'nin toplam ağırlığı derste söylenmedi. $1+2+2+4+4+7+8+9 = 37$. Python ile bütün 8 kenarlı alt kümeler denenerek en küçük kapsayan ağaç ağırlığının 37 olduğu doğrulandı; C kodu da 37 verdi.

Soru 12 (derste sözlü, 02:01:30): İkinci örnekte 0'ın değeri 1, 7'nin değeri 2 iken 0-7 kenarını almam gerekir mi?

> **Çözüm:** Evet. Değerler farklıdır; 0 ile 1'in grubu ve 2, 3, 5, 6, 7, 8'in grubu henüz ayrıdır. Bu kenar iki grubu birleştirir ve döngü oluşturmaz.

Dersin Toparlaması

Bu derste graflara giriş yapıldı. Grafın düğüm ve kenarlardan oluşan doğrusal olmayan bir veri yapısı olduğu, null, trivial, yönlü/yönsüz, bağlı, tam, döngülü, DAG, iki parçalı ve ağırlıklı graf türleri görüldü. Ağaçların grafların özel bir türü olduğu hatırlatıldı. Grafların komşuluk matrisi ve komşuluk listesiyle saklanabileceği, seçimin grafın yoğun ya da seyrek olmasına, yani kenar sayısına bağlı olduğu konuşuldu: seyrek grafa liste, yoğun grafa matris uygundur. Graflarda gezinmenin DFS ve BFS ile yapılacağı söylendi.

Dersin ikinci yarısında kapsayan ağacın bağlı ve döngüsüz iki şartı, bir graftan birden fazla kapsayan ağaç çıkabileceği ve bunlar içinde toplam ağırlığı en küçük olanın MST olduğu anlatıldı. MST'yi bulan iki açgözlü algoritmadan Kruskal ayrıntılı işlendi: kenarlar sıralanır, en küçükten başlanarak döngü oluşturmayan kenarlar eklenir; döngü kontrolü her düğümün küme değerini tutan `belongs` dizisiyle yapılır. Algoritma iki örnekte (MST ağırlıkları 15 ve 37) ve C kodu üzerinde izlendi. Bir sonraki derste Prim algoritması anlatılacak; belki en kısa yol için Dijkstra algoritması ya da başka bir graf algoritması da konuşulacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.